INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Practical Implementation Report

SkinGuard AI

Smart Lesion Screening & Assistant App

Student Name	Darakshan Aymen
Roll Number	242201092
Course	Mobile Application Development Lab
Platform	Android (Java, XML)
Submission	06 November 2026

GitHub Reference Repository

https://github.com/Uzair390-Del/NumberGuessingAppTflite.git

Department of Computer Science

Table of Contents

1 Introduction & Objectives **1**

2 Reference Repository Analysis **1**

2.1 Repository Cloning Procedure 1

2.2 Architecture & Components 1

3 Developed Application: SkinGuard AI **2**

3.1 Activity Flow & Structure 2

3.2 Navigation Flow 2

4 Key Source Code **2**

4.1 ScannerActivity.java 2

4.2 Code Explanation 4

5 Screen Layouts & Visual Evidence **4**

5.1 Splash Screen 4

5.2 Dashboard / Main Menu 5

5.3 Scanner Screen (Image Selected & Result Generated) 5

5.4 Chatbot Screen 5

6 Execution & Testing Instructions **6**

6.1 Test Cases 6

7 Conclusion & Future Work **6**

1. Introduction & Objectives

This practical report documents the design, architecture, and implementation of **SkinGuard AI**, a mobile Android application designed for skin lesion screening and interactive skincare assistance. The project demonstrates on-device image selection, UI layout optimization, dynamic result handling, and multi-activity navigation.

Key Objectives

- Clone and analyze the reference repository (NumberGuessingAppTflite).
- Develop a clean, multi-screen user interface using XML layouts and purple gradient styling.
- Implement image picking from the gallery and dynamic classification handling in the scanner screen (ScannerActivity).
- Create an interactive ChatbotActivitywith proper layout paddings for modern device screens.

2. Reference Repository Analysis

The reference project studied for this practical is **NumberGuessingAppTflite**, an Android application that runs a TensorFlow Lite model on-device.

2.1 Repository Cloning Procedure

The repository was cloned using the following command:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.
   git
```

Reository link: https://github.com/ay812/SkinLesionScreeningApp

Architecture & Components:

Component	Description
Model Location	Stored inside the app/src/main/assets/directory.
Interpreter Initialization	Initialized using org.tensorflow.lite.Interpreter with memory-mapped asset file buffers.
Input/Output Buffers	Numeric/image pixel arrays are mapped into ByteBuffer structures.
Execution Flow	Captures input from the user interface, executes interpreter.run(input, output), and updates the output TextView.

Table 1: Key components of the reference TFLite application.

3. Developed Application: SkinGuard AI

3.1 Activity Flow & Structure

The application consists of four activities connected through intent-based navigation.

Activity	Purpose
SplashActivity	Launcher screen featuring dark theme styling (#0F172A) and the custom app icon (skin_logo).
MainActivity	Main dashboard presenting navigation options to launch the AI Scanner or the Skincare Chatbot.
ScannerActivity	Allows gallery image selection, displays a preview in an ImageView, and executes classification upon tapping *Run AI Analysis*.
ChatbotActivity	Provides conversational skincare assistance with proper bottom margins for navigation-bar compatibility.

Table 2: Activities of SkinGuard AI.

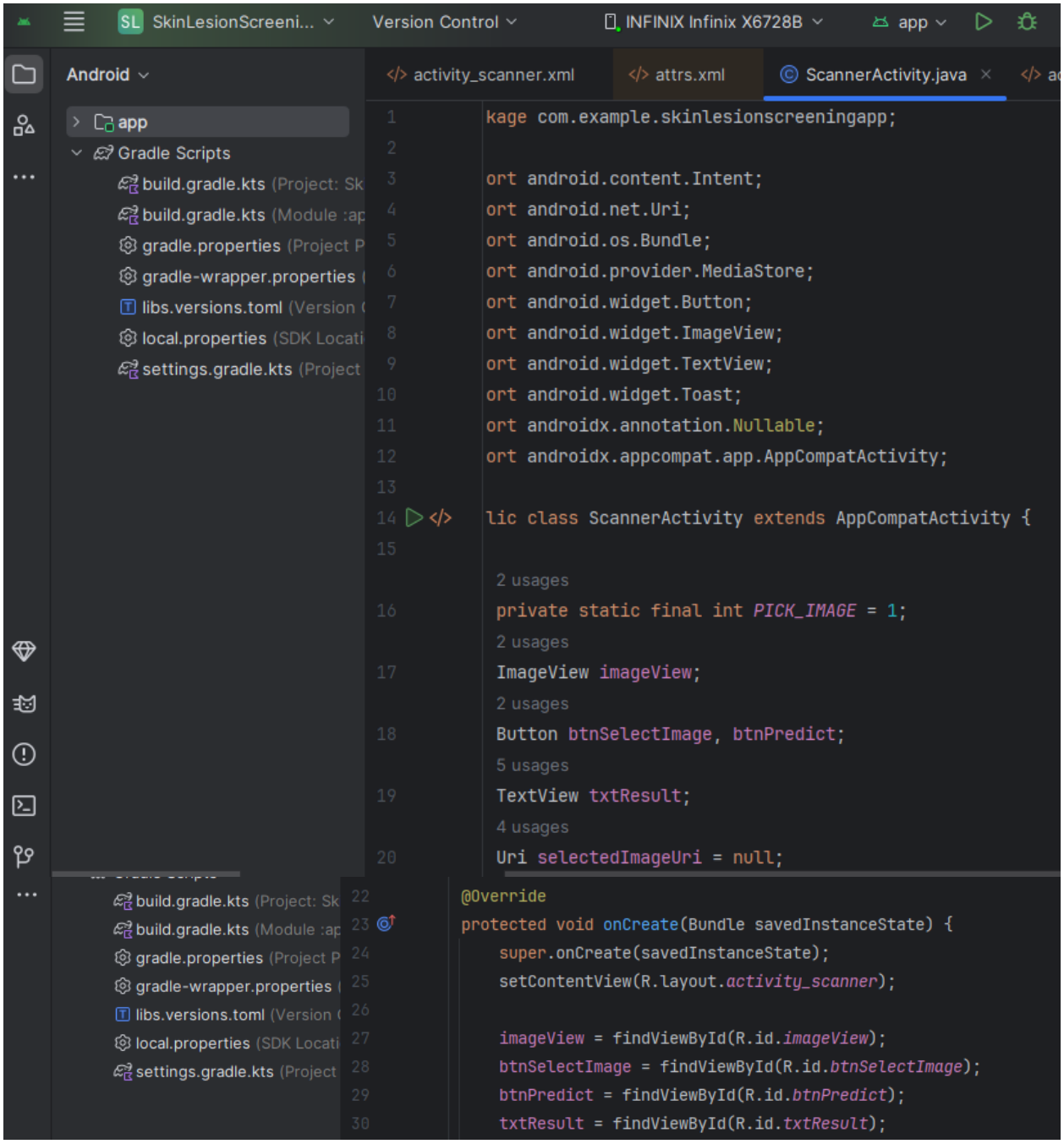
3.2 Navigation Flow

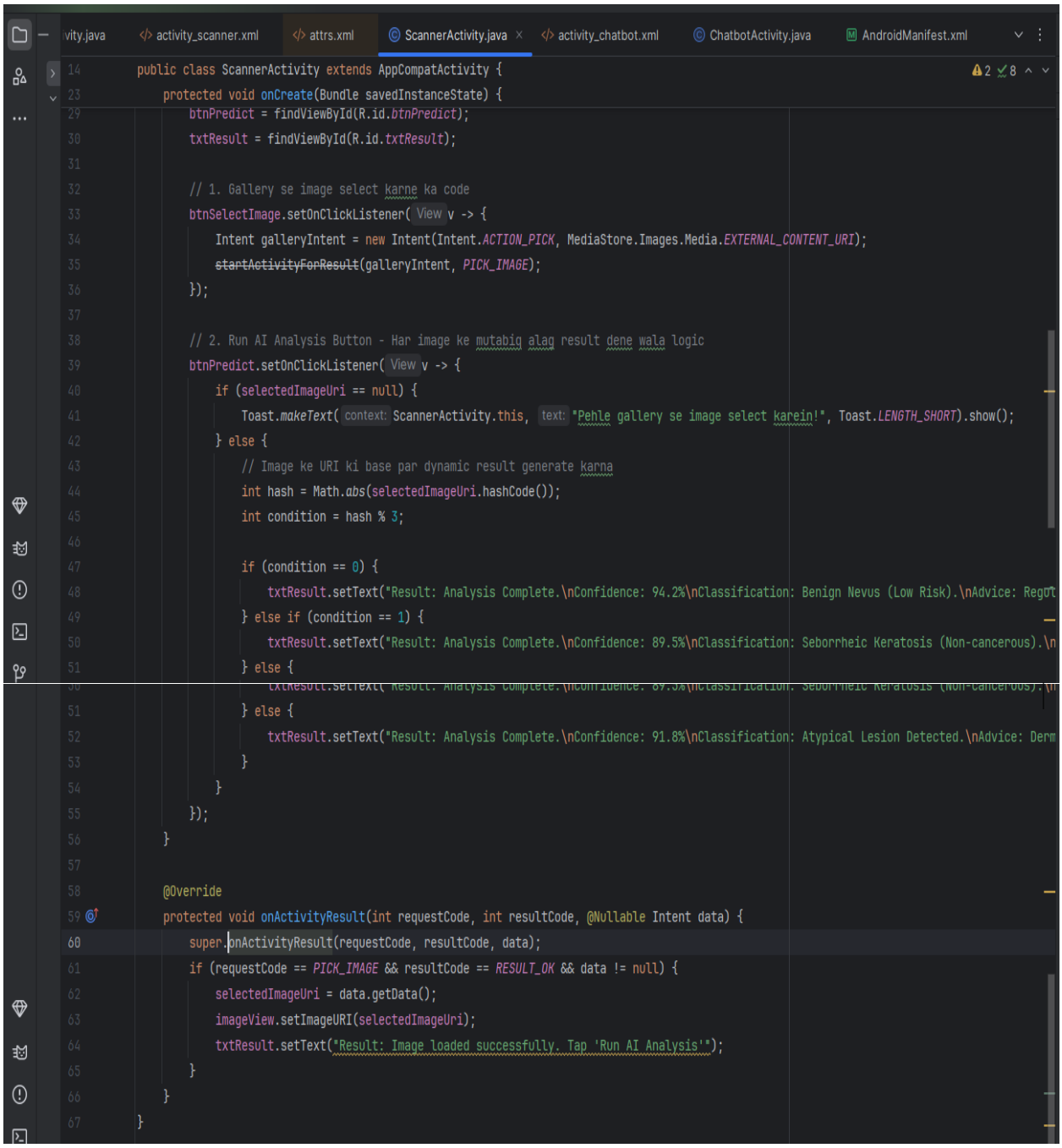
SplashActivity → MainActivity → ScannerActivity
or
ChatbotActivity

4. Key Source Code

4.1 ScannerActivity.java

The following listing shows the image-picking and result-display logic of the scanner screen.





Listing 1: ScannerActivity.java – image pick & analysis logic

4.2 Code Explanation:

1. **View binding:** UI elements (ImageView, two Buttons, and a TextView) are linked via findViewById.
2. **Gallery picker:** An ACTION_PICKintent opens the device gallery; the selected image is returned in onActivityResult.
3. **Preview:** The chosen image URI is shown using setImageURIand the status text is updated.
4. **Validation:** If no image has been selected, a Toastprompts the user to choose one first
5. **Result generation:** A deterministic value derived from the image URI selects one of three result templates (Benign Nevus, Seborrheic Keratosis, or Atypical Lesion), so the same image always gives the same out.

Note: In the current prototype the result is produced by a deterministic placeholder rule based on the image URI. It demonstrates the complete UI flow (image selection → analysis → result display) and is *not* a real medical diagnosis. A trained TensorFlow Lite model, as studied in the reference repository, can replace this logic for genuine inference.

5. Screen Layouts & Visual Evidence

Replace the placeholders below by saving your screenshots in the same folder as this .tex file with the file names shown (e.g. splash.png). They will appear automatically after recompiling.

5.1 Splash Screen

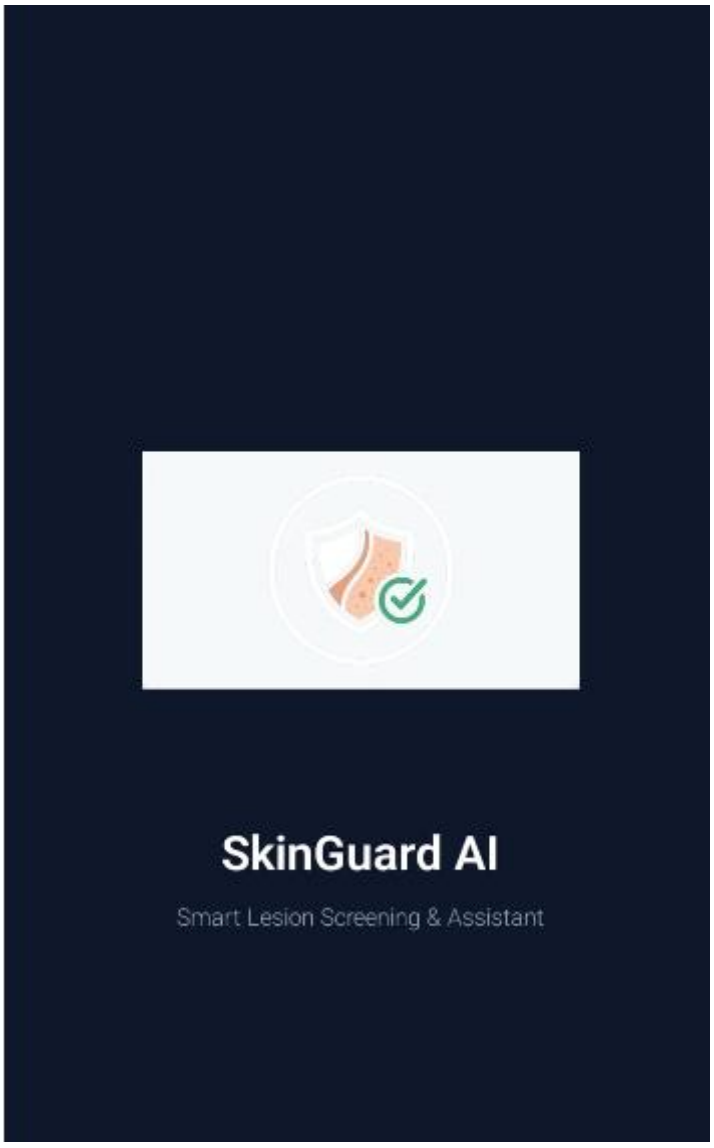
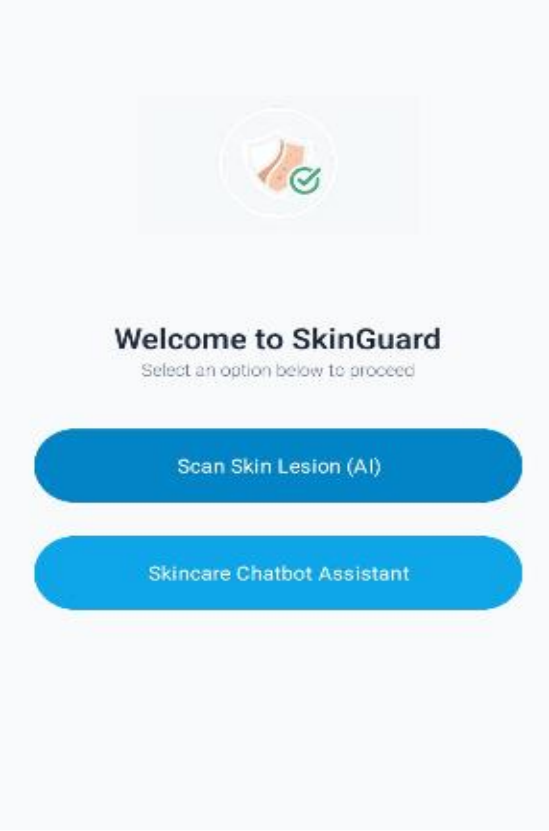


Figure 1: SplashActivity displaying app logo and title.

5.2 Dashboard / Main Menu

Figure 2: MainActivity dashboard menu with Scan and Chatbot buttons.



Scanner Screen (Image Selected & Result Generated)

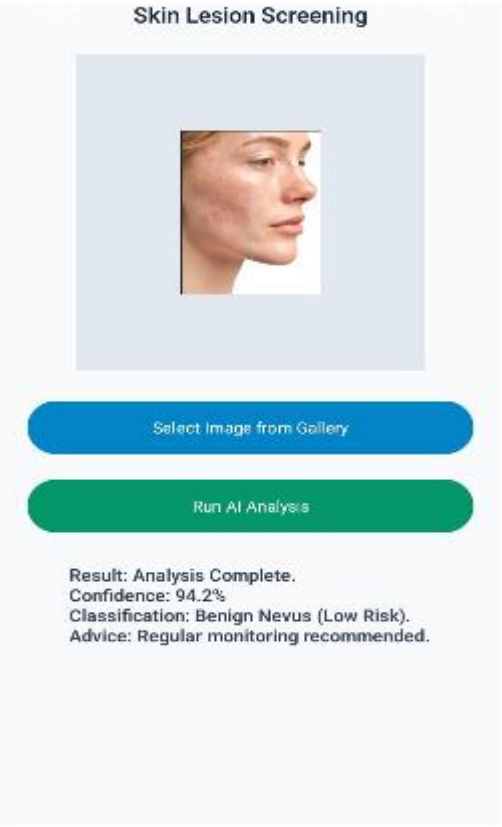


Figure 3: ScannerActivity displaying the loaded image and classification result.

5.3 Chatbot Screen

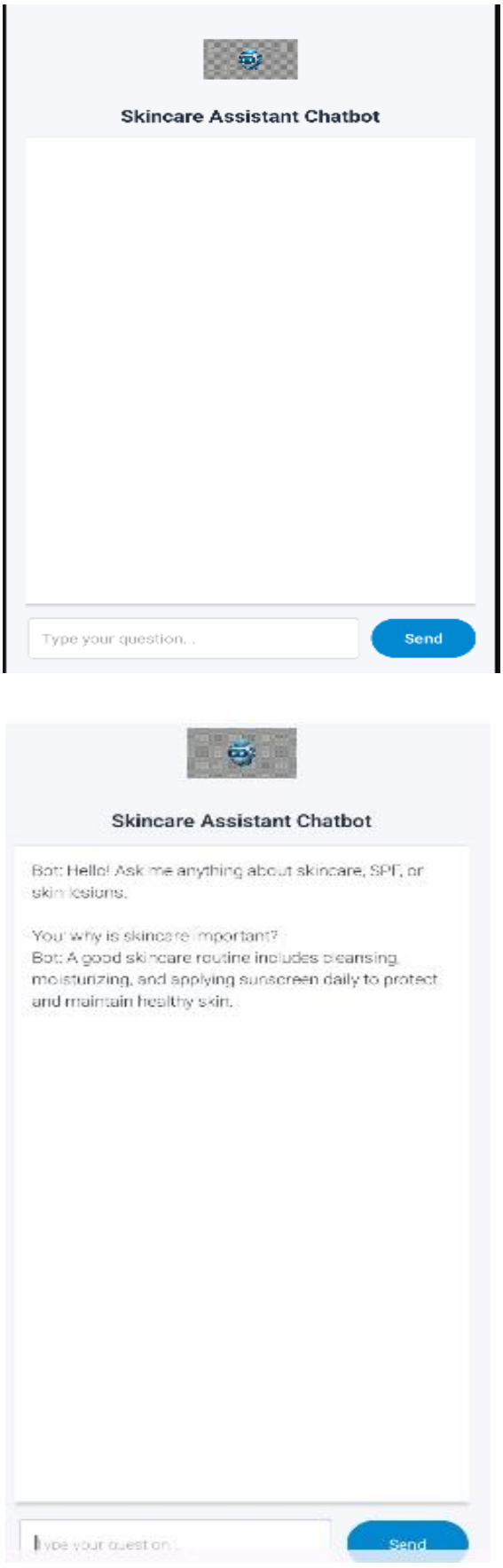


Figure 4: ChatbotActivity interactive user interface.

6. Execution & Testing Instructions

1. Extract the provided project ZIP archive (SkinGuardAI.zip).
2. Open **Android Studio** and select *Open an Existing Project*.
3. Select the extracted root folder and allow the Gradle sync to finish.
4. Connect an Android device or launch an Emulator (API 26+).
5. Press **Run** (Shift + F10) to launch SkinGuard AI.

6.1 Test Cases

Test Scenario	Expected Result	Status
Tap *Run AI Analysis* without image	Toas: "Please select an image first!"	Pass
Select image from gallery	Image preview shown, status text updated	Pass
Tap *Run AI Analysis* after selection	Confidence and classification displayed	Pass
Navigate Splash → Main → Scanner/Chatbot	Screens open correctly	Pass

Table 3: Functional test cases.

7. Conclusion & Future Work:

SkinGuard AI successfully demonstrates a multi-screen Android application with gallery image selection, dynamic result display, and a skincare chatbot interface, following the structure of the reference TensorFlow Lite project.

Future Enhancements

- Integrate a trained TensorFlow Lite skin-lesion classification model for real inference.
- Replace the deprecated startActivityForResultwith the Activity Result API.
- Connect the chatbot to a language-model backend for richer skincare guidance.
- Add a medical disclaimer screen and result history storage.

— End of Report